

```

        nextX = x - 2;
        nextY = y;
        maze[[x - 1, y]] = EMPTY; // Connecting hallway.
    } else if (nextIntersection === EAST) {
        nextX = x + 2;
        nextY = y;
        maze[[x + 1, y]] = EMPTY; // Connecting hallway.
    }
    hasVisited.push([nextX, nextY]); // Mark space as visited.
    visit(nextX, nextY); // Recursively visit this space.
}
}
}

```

The `unvisitedNeighbors` list or array contains one or more of the `NORTH`, `SOUTH`, `WEST`, and `EAST` constants. We choose one of these directions for the next recursive call to `visit()`, and then set the `nextX` and `nextY` variables with the coordinates of the neighboring space in this direction.

After this, we add the `x`, `y` coordinates of `nextX` and `nextY` to the `hasVisited` list or array before making the recursive call for this neighboring space. In this way, the `visit()` function continues to visit neighboring spaces, carving out the maze hallways by setting locations in `maze` to `EMPTY`. The connecting hallway between the current space and neighboring space is also set to `EMPTY`.

When no neighbors exist, the base case simply returns to an earlier location. In the `visit()` function, the execution jumps back to the start of the `while` loop. The code in the `while` loop again checks which neighboring spaces haven't been visited and makes a recursive `visit()` call on one of them, or returns if all neighboring spaces have already been visited.

As the maze fills up with hallways and each space has been visited, the recursive calls will continue to return until the original `visit()` function call returns. At this point, the `maze` variable contains the completely generated maze.